OOV Handler (Out-of-Vocabulary Handler)

Where this fits in the project

The complete sign-language translation system consists of four main stages. First, speech or text is received from the user. Next, the **OOV (Out-of-Vocabulary) Handler** detects words that are unfamiliar or outside the language patterns expected by the system and rewrites them into simpler or semantically equivalent English words. The rewritten sentence is then passed to the **Gloss Generator**, which converts it into gloss compatible with the sign-language vocabulary. Finally, the **Sign Mapping** module translates the gloss into the corresponding sign animations or recognition output.

In the overall architecture, the OOV Handler acts as the first Natural Language Processing (NLP) component and prepares user input before gloss generation.

Speech/Text
↓
OOV Handler
↓
Gloss Generator
↓
Sign Mapping

What this does

Users naturally use words that may not directly correspond to the concepts understood by the sign-language translation pipeline. For example, they may write:

"I want a sandwich."

Although **sandwich** is a perfectly valid English word, later stages of the system may not represent that exact concept directly.

Instead of failing or forcing the user to rephrase the sentence, the OOV Handler rewrites unfamiliar words into semantically similar English words while preserving the original meaning as closely as possible.

For example:

Original sentence

I want sandwich.

Rewritten sentence

I want food.

Another example:

Original sentence

Mom and dad are here.

Rewritten sentence

Parents are here.

These rewritten sentences are then passed to the Gloss Generator, which performs the actual conversion into sign-supported gloss.

Unlike traditional OOV systems, this module **does not restrict replacements to the sign vocabulary**. Its responsibility is semantic rewriting only. Vocabulary adaptation happens later in the Gloss Generator.

How it works

The module resolves unknown words using four fallback layers, ordered from highest quality to highest reliability.

1. Detect unknown words

The input sentence is first tokenized into individual words.

Each word is compared against the system's known vocabulary list.

Words already recognized are left unchanged.

Unknown words are collected for further processing.

For example,

Input

I want sandwich

Detected OOV words

["sandwich"]

---

2. Ask the local language model

Each unknown word is sent to a locally running Large Language Model (LLM) through Ollama.

Instead of selecting words from a predefined vocabulary, the model is instructed to replace the unknown word with a simple English word or short phrase that preserves the original meaning.

The prompt instructs the model to:

- preserve the original meaning
- prefer common English words
- avoid unnecessary explanations
- return only a JSON array
- keep replacements short (usually one word)

For example,

Input

hospital

Possible output

["clinic"]

Input

sandwich

Possible output

["food"]

Input

mom

Possible output

["mother"]

Because the LLM understands semantic relationships, it usually produces the most natural replacement.

---

3. Semantic similarity fallback

If the LLM is unavailable or produces an invalid response, the module falls back to semantic similarity using Sentence Transformers.

The unknown word and a collection of candidate English words are converted into embeddings.

Cosine similarity is then used to find the closest semantic match.

This layer requires no LLM and works completely offline once the embedding model has been downloaded.

Although generally less accurate than the LLM, it can still recover many common words.

---

4. Manual synonym dictionary

If semantic matching also fails, the module consults a manually curated synonym dictionary.

Unlike earlier versions of the project, dictionary entries **are not limited to the sign vocabulary**.

Instead, each entry simply contains one or more English words that preserve the original meaning.

For example,

"hospital" → ["clinic"]
"pizza" → ["food"]
"mom" → ["mother"]
"dad" → ["father"]

This layer is extremely fast because it performs only a dictionary lookup.

---

5. Keep the original word

If every previous layer fails, the original word is preserved.

For example,

Input

blockchain

Output

blockchain

This guarantees that the translation pipeline never crashes or silently removes information from the user's sentence.

Example runs

Original sentence	Rewritten sentence
I want sandwich	I want food
Hospital go	Clinic go
Mom and dad	Mother and father
I need pizza	I need food
The nurse helped me	The medical worker helped me

The rewritten sentence is then passed directly to the Gloss Generator for further processing.

Why Ollama instead of a paid API

The module uses Ollama to run the language model locally.

Running locally provides several advantages:

- no API key is required
- no internet connection is needed after downloading the model
- no usage cost
- user data remains on the local machine

The tradeoff is that smaller local models may occasionally generate weaker semantic replacements than larger cloud-based models. This is why the module includes multiple fallback layers.

Setup

1. Install Ollama

https://ollama.com/download

2. Download a model

ollama pull phi3:mini

or

ollama pull qwen2.5:7b

for better quality if your hardware supports it.

3. Install the required packages

pip install ollama
pip install sentence-transformers

4. Run the module

python oov_handler.py

What's in the file

TARGET_FOLDERS / VOCAB_SET

Stores the known vocabulary used to detect out-of-vocabulary words.

OOVResult

A result object containing:

- original words
- rewritten words
- replacement method
- confidence score
- warnings
- replacement mappings

---

find_oov_words()

Identifies which words are outside the known vocabulary.

---

build_system_prompt() / build_user_prompt()

Constructs the prompts sent to the local language model.

_call_ollama()

Communicates with the local Ollama model and retrieves replacement suggestions.

---

parse_response()

Parses and validates the JSON returned by the language model.

---

validate_mapping()

Ensures the replacement list is valid by checking that it contains non-empty strings and does not exceed the maximum allowed length.

---

llm_fallback()

Attempts to resolve unknown words using the local language model.

---

semantic_fallback()

Finds semantically similar replacements using sentence embeddings.

---

dictionary_fallback()

Looks up manually curated synonym replacements.

---

resolve_single_word()

Coordinates the four-layer resolution strategy.

---

replace_words()

Reconstructs the sentence using the resolved replacements.

---

process_oov()

The main function used by the rest of the project.

It detects unknown words, resolves them using the layered fallback strategy, rebuilds the sentence, and returns an `OOVResult` object.

Using it in code

```
from oov_handler import process_oov

result = process_oov(["I", "want", "sandwich"])

print(result.mapped_words)
print(result.method)
print(result.confidence)
print(result.warnings)
```

Example output

['I', 'want', 'food']
llm_local
1.0
[]

Known limitations

- Small local language models may occasionally produce replacements that are too general or slightly alter the intended meaning.
- Semantic similarity is based on word embeddings and can occasionally choose an incorrect synonym for ambiguous words.
- The manual dictionary requires ongoing maintenance as new words are encountered.
- Very domain-specific or technical terminology may remain unresolved and be passed through unchanged.
- Processing with a local LLM is slower than a simple dictionary lookup, especially on CPU-only systems.